

AI-Driven Supply Chain Disruption Prediction and Mitigation Platform

PHASE I REPORT

Submitted by

SHANKARANARAYANAN 230701306

D. SAI KHUSHEE 230701276

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

December 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled “**AI-DRIVEN SUPPLY CHAIN DISRUPTION PREDICTION AND MITIGATION PLATFORM**” is the bonafide work of **SHANKARANARAYANAN (230701306), D. SAI KHUSHEE (230701276)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

“This project addresses the following Sustainable Development Goal(s): Goal No. 9 – Industry, Innovation and Infrastructure.”

SIGNATURE

Dr. J. Manoranjini,
Professor and Head,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai - 602 105.

SIGNATURE

Dr. P. Kumar,
Professor,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai - 602 105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

Car production depends on electronic parts that pass through several layers of suppliers. A car maker generally knows only its direct suppliers, so the chip makers, foundries and material vendors below them remain invisible. The 2021–2022 chip shortage showed the cost of this blind spot: many car plants stopped work because of parts they had not realised they relied on. Large companies buy expensive mapping and monitoring platforms specialized to their supply chain, whereas small and mid-sized automotive electronics manufacturers cannot buy or build such systems. This project constructs an AI platform that gives them the same protection at low cost. The platform consists of three parts. First, a graph neural network predicts hidden supplier links from partial data, and its robustness is tested by deleting known links step by step (10–50%) and measuring accuracy at each level. Second, a disruption alerting module reads news and global event data, turns each article into a structured disruption event with a locally run language model, links it to the company’s own supply network, and raises an alert only after multiple independent sources confirm the same event, which cuts out false alarms. Third, a mitigation module recommends responses, checks every recommendation against the evidence it cites, and acts on its own only when its confidence crosses a set threshold. Based on published GraphSAGE results on a comparable supply network, the prediction model is projected to retain about 84% of its accuracy when half of the known links are hidden; a first prototype run on the real CSET/ETO chip supply dataset measured 93% retention. Phase I also delivers the first stage of the alerting part: on live news, the extraction module correctly picked out a real plant halt at a chip supplier, linked it to the buyers in the uploaded network, and dismissed the unrelated articles. The platform is built on public data – the CSET/ETO chip supply dataset, public supplier records, news articles and the GDELT event database – and will be validated by replaying the 2021–2022 chip shortage.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavour to put forth this report. Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E., F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavouring in educating us in their premier institution.

Our sincere thanks to **Dr. S. N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. MANORANJINI, Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work.

We convey our sincere and deepest gratitude to our internal guide, **Dr. P. KUMAR**, Professor, Department of Computer Science and Engineering, for his valuable guidance throughout the course of the project. We thank our Project Coordinator, **Dr. S. MADHUSUDHANAN**, Associate Professor, Department of Computer Science and Engineering, for his useful tips during our reviews to build our project.

SHANKARANARAYANAN : 230701306

D. SAI KHUSHEE : 230701276

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	3
	ACKNOWLEDGEMENT	4
	LIST OF FIGURES	7
	LIST OF TABLES	7
1	INTRODUCTION	8
	1.1 Overview	8
	1.2 Problem Statement	9
	1.3 Objectives of the Project	9
	1.4 Scope of Phase I	10
	1.5 Organisation of the Report	10
2	LITERATURE SURVEY	11
	2.1 Link Prediction and Graph Neural Networks	11
	2.2 Graph Learning on Supply Networks	11
	2.3 Supply Chain Disruption and Event Data	12
	2.4 Language Models for Event Extraction	12
	2.5 Research Gaps	13
3	PROPOSED WORK	14
	3.1 Overview of the Proposed System	14
	3.2 Objectives of Phase I	14
	3.3 System Architecture	15
4	IMPLEMENTATION AND RESULTS DISCUSSION	17
	4.1 Datasets	17
	4.2 Tools and Environment	18
	4.3 Execution Flow	18
	4.4 Algorithms	19
	4.5 Key Functions	20
	4.6 Module Description	21
	4.6.1 Module 1 - Data Ingestion and Graph Construction	21
	4.6.2 Module 2 - GNN Hidden Link Prediction	21

4.6.3 Module 3 - Visibility-Degradation Robustness Protocol	22
4.6.4 Tierline Web Platform	22
4.6.5 Module 4 - NLP Disruption Signal Extraction	23
4.6.6 Phase II Modules (5 and 6)	24
4.7 Measured Results on the Real ETO Graph (Modules 1-3)	25
4.8 Effect of the Asymmetric Decoder (Tierline)	26
4.9 Disruption Signal Extraction Results (Module 4)	26
4.10 Projected Results for the Full Five-Method Sweep	28
4.11 Platform Verification	29
4.12 Discussion	33
5 CONCLUSION AND FUTURE WORK	35
5.1 Conclusion	35
5.2 Future Work - Phase II Plan	35
REFERENCES	37
CONFERENCE/JOURNAL PAPER PUBLICATION	39

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	Overall architecture of the proposed platform	15
3.2	Hidden-link prediction pipeline (Modules 1–3)	16
3.3	Disruption signal extraction pipeline (Module 4)	16
4.1	Measured test AUC on the real ETO graph (NumPy prototype)	25
4.2	Effect of the asymmetric decoder on the Tierline sample network	26
4.3	Projected AUC degradation curves for all methods	28
4.4	Tierline platform screens – LOGIN	30
4.5	Tierline platform screens – UPLOAD	30
4.6	Tierline platform screens – CONFIRM	31
4.7	Tierline platform screens – RESULTS (a)	31
4.8	Tierline platform screens – RESULTS (b)	32
4.9	Tierline platform screens – Disruption Signal Extraction	32
4.10	Tierline platform screens – NewsAPI Output	33

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
4.1	Tools and libraries used	18
4.2	Key functions of the Phase I implementation	20
4.3	Module ownership and status	21
4.4	Tierline upload format	23
4.5	Measured results on the real ETO graph	25
4.6	Module 4 results on live news and bundled samples	27
4.7	Projected test AUC under increasing visibility degradation	28

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

A modern car contains between one and three thousand chips. They control the engine, brakes, airbags, windows and displays. These chips reach the car maker through a long chain: material vendors, wafer foundries, chip makers, packaging firms and tier-1 electronics suppliers. Most car makers keep good records only for their tier-1 suppliers. The layers below stay out of sight, so the risks in those layers stay out of sight too.

The 2021–2022 chip shortage showed the cost of this blind spot. Many car makers, including some of the largest, had to halt production because of parts they did not know they depended on [11]. Large car makers can buy commercial supply chain mapping tools or run long manual mapping projects. Small and mid-sized automotive electronics manufacturers cannot afford either. For them, a method that infers likely hidden supplier links from the partial data they already have is far more useful.

Link prediction does exactly this. Given a graph of known supplier–buyer links, it predicts which unseen links probably exist. Recent work shows that graph neural networks (GNNs) do this well on supply networks [9]. But there is a basic issue with the way these models are tested. The whole point of the task is that real supply graphs are incomplete. Yet the models are usually trained and tested on graphs treated as complete. A model that fails when many edges are missing is ineffective in the exact situation it was built to handle.

Knowing the network is only half of the problem. A fire at a chip fab or an earthquake near a foundry is reported in the news within hours, but a small manufacturer has no simple way to tell whether a given story touches its own suppliers. Reading every article by hand does not scale, and keyword alerts fire on stock reports and product launches as often as on real disruptions.

This project builds an AI-driven platform that addresses the problem end to end: it predicts the hidden structure of a company’s supply network, reads public news and event data for disruptions that touch that network, and recommends mitigation. The work is split across two semesters. Phase I, reported here, delivers the prediction half of the platform and the first module of the alerting half – extraction of structured disruption

signals from news, linked to the company’s own network. Phase II delivers corroborated alerting, mitigation and the integrated dashboard.

1.2 PROBLEM STATEMENT

Small and mid-sized automotive electronics manufacturers have no affordable way to see beyond their tier-1 suppliers. Existing link-prediction studies on supply networks assume the observed graph is complete and do not measure how prediction quality falls as visibility falls. Classical heuristics such as the Jaccard coefficient and Adamic–Adar depend on visible neighbourhood overlap, so they should fail precisely when visibility is poor, but this has not been measured on supply chain data. As a result, a manufacturer has no way to know how far it can trust a predicted hidden link when its own records are incomplete. In the same way, it has no low-cost way to turn the daily flow of news into a short list of events that actually affect the companies in its own network.

1.3 OBJECTIVES OF THE PROJECT

The overall project has the following objectives. Objectives (a)–(e) are the scope of Phase I; (f)–(g) are the scope of Phase II.

- (a)Build a directed automotive electronics supply graph from public data: the CSET/ETO advanced chip supply chain dataset plus public automotive supplier records.
- (b)Train a GraphSAGE encoder with a direction-aware link decoder that predicts hidden supplier–buyer links from a partial graph.
- (c)Define and run a visibility-degradation protocol that removes 10–50% of known training edges and reports a single retention metric, comparing GraphSAGE with Jaccard, Adamic–Adar, node2vec and GCN.
- (d)Deliver a web platform through which a company uploads its own supplier list and receives ranked hidden-link predictions with a tier-stratified network view.
- (e)Extract structured disruption signals from news articles with a locally run language model, check each signal against the article text, and link the affected companies to the uploaded supply graph.

- (f) Add the GDELT event stream as an independent source and raise disruption alerts only after multiple independent sources corroborate an event (Phase II).
- (g) Recommend mitigation actions that are checked against cited evidence and gated by a confidence threshold, and integrate all modules in one dashboard (Phase II).

1.4 SCOPE OF PHASE I

Phase I covers Modules 1, 2, 3 and 4 of the platform (data ingestion and graph construction, GNN hidden-link prediction, visibility-degradation robustness, and NLP disruption signal extraction) and the Tierline web platform. The GNN prototype is implemented in NumPy with backpropagation verified against numerical gradients; the full PyTorch Geometric sweep with five baselines is the first task of Phase II. Module 4 runs Llama 3.1 on the team’s own laptop through Ollama and gathers articles through NewsAPI; it is tested end to end on live news, but a labelled test set for an accuracy figure is a Phase II task. Measured results in this report come from the real ETO dataset and from live runs of the platform; results for the full five-method sweep are projections anchored in published GraphSAGE performance [9] and are labelled as such wherever they appear.

1.5 ORGANISATION OF THE REPORT

Chapter 2 surveys the literature on link prediction, graph neural networks, supply chain risk and language-model event extraction, and summarises the gaps. Chapter 3 describes the proposed work and the system architecture. Chapter 4 covers the implementation – datasets, tools, algorithms, key functions and each module – and presents the measured and projected results with a discussion. Chapter 5 concludes and lays out the Phase II plan. References and the paper publication details follow.

CHAPTER 2

LITERATURE SURVEY

2.1 LINK PREDICTION AND GRAPH NEURAL NETWORKS

Liben-Nowell and Kleinberg [1] defined the link prediction problem and showed that simple neighbourhood scores such as common neighbours, the Jaccard coefficient and Adamic–Adar work well on social networks. These scores rate a pair of nodes only by the overlap of their visible neighbours. This makes them cheap, but it also ties them completely to how much of the graph is visible. Grover and Leskovec [2] introduced node2vec, which learns node embeddings from biased random walks and predicts links from embedding similarity; it captures some longer-range structure but still needs walks through visible edges.

Graph neural networks made this learning end-to-end. The GCN of Kipf and Welling [3] mixes neighbour features through graph convolutions, and Kipf and Welling [13] extended it to a variational graph auto-encoder for link prediction. GraphSAGE by Hamilton et al. [4] learns an aggregation function instead of one fixed embedding per node, so it also works for nodes and edges not seen in training – the inductive property this project relies on. The GAT of Veličković et al. [5] weighs neighbours with attention, and SEAL by Zhang and Chen [6] learns from small enclosing subgraphs around each candidate pair. Wu et al. [14] give a broad survey of GNN architectures and their applications, including link prediction.

2.2 GRAPH LEARNING ON SUPPLY NETWORKS

In the supply chain field, Baryannis et al. [7] reviewed artificial intelligence for supply chain risk management and noted the lack of data-driven prediction methods. Brintrup et al. [15] first framed hidden-link prediction in supply networks as a machine-learning problem using engineered graph features. Brintrup et al. [16] then used supplier-level data analytics to predict supplier disruptions in complex asset manufacturing, showing that firm-level data can carry a predictive signal. Wichmann et al. [8] built supply chain maps automatically from news text, which showed that such maps can be made from public data but are always partial.

Closest to this work, Kosasih and Brintrup [9] applied GraphSAGE to an automotive supply network, the same industry studied here, and reported strong accuracy

in finding hidden supplier links. This is the base paper for the project. Aziz et al. [10] studied data-preparation choices for graph representation learning on supply networks, and Kosasih et al. [17] extended the approach to knowledge-graph reasoning for supply chain risk management with GNNs.

2.3 SUPPLY CHAIN DISRUPTION AND EVENT DATA

Mohammad et al. [11] analysed the causes of the 2021–2022 semiconductor chip shortage and its heavy impact on car production, which motivates the automotive electronics focus of this project. Ivanov and Dolgui [18] introduced the viability perspective on intertwined supply networks, and Ivanov [19] showed how epidemic-driven disruptions propagate through global supply chains, both of which argue for continuous monitoring rather than one-time mapping. Leetaru and Schrodtt [20] describe the GDELT event database, the open global news-event stream that the Phase II alerting module will consume. The Emerging Technology Observatory [12] publishes the Supply Chain Explorer dataset on advanced chips, which is the primary public data source used in Phase I.

2.4 LANGUAGE MODELS FOR EVENT EXTRACTION

Turning a news article into a structured record – what happened, where, to whom and how badly – used to need task-specific models trained on labelled text, as in the supply-chain map extraction of Wichmann et al. [8]. Open large language models now do much of this from an instruction and a fixed output schema. The Llama 3 family [21] includes an 8-billion-parameter model that runs on a laptop and can be asked to answer in strict JSON, which makes structured extraction possible without sending a company’s data to a paid cloud service. The known weakness of such models is hallucination: they can produce fluent output that is not supported by the source text. Ji et al. [22] survey this problem across summarisation, question answering and data-to-text tasks and show that checking output against the source is one of the main ways to reduce it. For a supply chain tool this matters directly, because a company name invented by the model would send an alert to the wrong supplier.

2.5 RESEARCH GAPS

Four gaps follow from this survey. First, existing supply chain link prediction studies, including [9], test models on the observed graph as if it were the full truth. None of them measures, step by step, how accuracy falls as visibility falls, even though missing visibility is the whole reason for the task. Second, the classical heuristics used as baselines depend on visible neighbourhood overlap, so they should break down exactly when they are needed most; this has not been measured on supply chain data. Third, the automotive electronics chain, where hidden dependencies caused the largest recent disruption [11], has been used in only one prior study [9], and that study did not test robustness. Fourth, small open language models can turn news into structured disruption records cheaply, but their output must be checked against the article before it is linked to real suppliers [22]; no low-cost tool does this for small manufacturers. Beyond prediction and extraction, no low-cost platform ties hidden-link prediction to corroborated disruption alerts and evidence-checked mitigation for small and mid-sized manufacturers.

CHAPTER 3

PROPOSED WORK

3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed platform has three parts: a hidden-link predictor, a disruption alerting module and a mitigation module, built on public data and delivered through a web application. Phase I builds and tests the predictor, whose robustness under partial visibility is measured directly instead of assumed, and the first stage of the alerting part. The Phase I work aims to (a) build a directed automotive electronics supply graph from the CSET/ETO chip data plus public automotive supplier records, (b) train a GraphSAGE encoder with a direction-aware MLP link decoder on versions of this graph with 0%, 10%, 20%, 30%, 40% and 50% of the training edges removed, (c) score every version on the same held-out test edges using AUC and average precision, (d) report robustness as one retention number, (e) compare the model with Jaccard, Adamic–Adar, node2vec and GCN under the same protocol, (f) expose the predictor through a web platform that works on a company’s own upload, and (g) convert news articles into structured disruption signals that are checked against the article text and linked to the uploaded network.

3.2 OBJECTIVES OF PHASE I

- Construct a cleaned, directed supply graph $G(V, E)$ with per-node feature vectors from public data.
- Implement a GraphSAGE link predictor whose decoder distinguishes “A supplies B” from “B supplies A”.
- Implement the visibility-degradation protocol and retention metric, and measure GraphSAGE against heuristic baselines on the real ETO graph.
- Deliver the Tierline web platform: CSV upload, graph confirmation, per-upload training, ranked predictions and tier-stratified visualisation, verified end to end.
- Implement Module 4: gather news, structure each article with a local language model, reject companies the article does not name, link the rest to the uploaded graph and list the directly exposed buyers.
- Submit the Phase I research paper to a conference (accepted at SMART-2026).

3.3 SYSTEM ARCHITECTURE

Figure 3.1 shows the architecture of the full platform. Data flows from top to bottom. The data layer has five sources: the CSET/ETO chip supply dataset, public automotive supplier records, a company’s own CSV upload through Tierline, news articles gathered through NewsAPI or pasted by the user, and – in Phase II – the GDELT event stream. Phase I modules are shown as solid boxes; Phase II modules are grey.

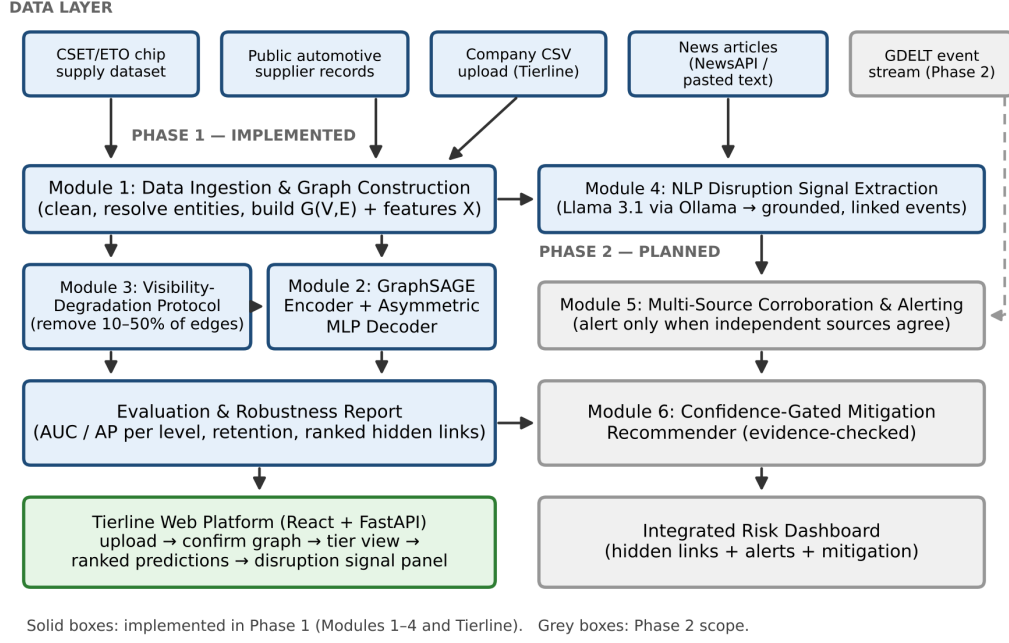


Figure 3.1 Overall architecture of the proposed platform

Module 1 cleans the records and builds the directed graph and node feature matrix. Module 3 removes a chosen share of the training edges to simulate a manufacturer’s limited visibility, and Module 2 trains the GraphSAGE encoder and asymmetric decoder on what remains. The evaluation block scores every configuration on the same held-out test edges, computes the retention metric and outputs the ranked list of predicted hidden links. Module 4 reads news articles, turns each one into a structured disruption event and links the companies it names to the graph built by Module 1, so every signal comes with the list of buyers in the company’s own network that are directly exposed. Tierline wraps the prediction pipeline and the signal panel in one web application. In Phase II, Module 5 adds GDELT as a second independent source and raises an alert only when independent sources agree, and Module 6 recommends mitigation whose every claim is checked against the evidence it cites. The predicted

hidden links feed the risk scoring of the alerting layer, which is why the two halves are designed together.

Figure 3.2 shows the Phase I prediction pipeline in more detail. One configuration file controls every run, so any experiment can be repeated exactly.

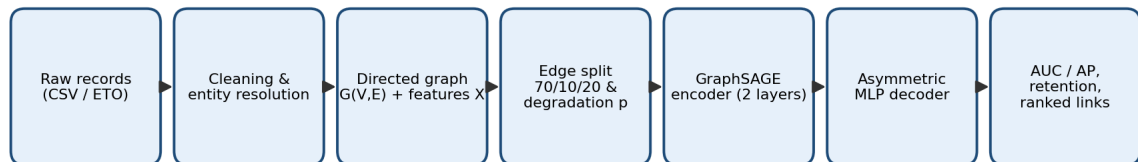
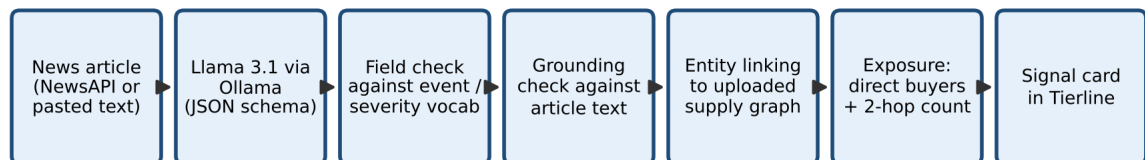


Figure 3.2 Hidden-link prediction pipeline (Modules 1–3)

Figure 3.3 shows the Module 4 pipeline. An article enters from NewsAPI or is pasted into the platform. The local model returns one JSON record per article; the record is checked field by field, every company in it is checked against the article text, and only the companies that pass are linked to nodes in the uploaded graph. The last step lists who buys directly from each affected company, which is the hand-off point for the Phase II corroboration module.



Fallback if Ollama is unreachable: Anthropic API → rule-based lexicon. Every result records which path produced it.

Figure 3.3 Disruption signal extraction pipeline (Module 4)

CHAPTER 4

IMPLEMENTATION AND RESULTS DISCUSSION

4.1 DATASETS

The graph combines two public sources. The first is the Supply Chain Explorer data from the Emerging Technology Observatory (ETO) of Georgetown University’s Center for Security and Emerging Technology (CSET) [12], available at github.com/georgetown-cset/eto-chip-explorer. It maps the advanced chip supply chain: materials, tools, process stages and the firms and countries that provide them. From this data the parts relevant to automotive chips – microcontrollers, power semiconductors and sensors – are kept. The processed ETO graph used in Phase I has 282 companies and 720 reported supply relationships. The second source is a set of automotive supplier records built from public company disclosures and news reports, in the spirit of [8], which adds car makers and their tier-1 electronics suppliers as nodes.

Module 4 reads text rather than graph records. Live articles come from the NewsAPI.org /v2/everything endpoint, queried with a company name plus disruption terms; the title, description and available body text of each article are joined into the text the model reads. On the free plan articles arrive with a 24-hour delay, body text is cut to about 200 characters and the key allows 100 requests a day. The platform also ships five sample articles written by the team in original wording – a fab fire, an earthquake, a port strike, a new export-licence rule and a quarterly-results control that contains no disruption – so the module can be shown without internet access.

Data caveat. The ETO data records which companies provide which tools, materials and process stages; it does not contain firm-to-firm relationships. The Phase I measured results therefore predict hidden company → process links (for example, which tool a company is likely also to make), not company → company links. Firm-level automotive relationships (MarkLines, FactSet Supply Chain Relationships, Bloomberg SPLC, Mergent) are paid-only. The automotive sample network bundled with Tierline (56 companies, 175 relationships, four tiers) uses real company names, but its specific edges are constructed to give a realistic multi-tier topology and are not verified commercial relationships. Predictions from that file, and the exposure lists Module 4 draws from it,

demonstrate the workflow and are never presented as empirical findings. The empirical claim of Phase I rests on the ETO experiment.

4.2 TOOLS AND ENVIRONMENT

Table 4.1 Tools and libraries used

Component	Tool / library	Purpose
Language	Python 3.11	Data processing, model, back end
GNN prototype	NumPy	GraphSAGE forward/backward pass, verified against numerical gradients
Planned GNN	PyTorch 2.3, PyTorch Geometric 2.5	Phase II full sweep with GCN, node2vec baselines
Graph handling	NetworkX 3.3	Graph construction, heuristic baselines
Data	pandas, NumPy	Loading, cleaning, entity resolution
Metrics	scikit-learn	AUC-ROC, average precision
Plots	Matplotlib	Degradation curves, results charts
Local LLM	Ollama 0.16.2, Llama 3.1 8B	Structured disruption extraction on the laptop GPU (Apple M3, Metal)
LLM fallback	Anthropic API (optional)	Used only when the Ollama server is not reachable
News source	NewsAPI.org	Gathering recent articles for a company
HTTP client	httpx	Calls to Ollama and NewsAPI
Back end	FastAPI, Uvicorn	Upload, training, prediction and signal API
Front end	React (Vite)	Upload, confirmation, results and signal panel
Testing	Playwright, pytest	End-to-end browser automation; 15 Module 4 tests
Execution	Google Colab (T4 GPU) / MacBook Air (Apple M3)	Experiments and demo

4.3 EXECUTION FLOW

Execution is script-driven. `preprocess.py` builds and saves the graph; `train.py --drop 0.3 --seed 1` trains one configuration; `run_all.sh` sweeps all six drop rates and five seeds (30 runs per model) and appends every result to `results.csv`; `plots.py` turns the CSV into the figures. `real_experiment.py` reproduces the measured ETO results reported in Section 4.7, and `sanity_check.py` prints live predictions for a chosen company. For the platform, the React front end is built once and served by the FastAPI back end so the demo runs as one process on one URL.

Module 4 needs two processes. The first terminal runs `OLLAMA_KEEP_ALIVE=-1 ollama serve`, which keeps Llama 3.1 loaded on the GPU; the first load takes about 16 seconds and later calls do not reload the model. The second terminal exports `NEWSAPI_KEY`, sends one warm-up request to the model and then

starts uvicorn main:app --port 8000 from the backend folder. The module can also be run without the web front end: run_demo.py --news "Renesas Electronics" --mode ollama gathers, structures and links live articles, and run_demo.py --mode ollama runs the bundled sample articles fully offline.

4.4 ALGORITHMS

Algorithm 4.1 Visibility-Degradation Evaluation Protocol

```

Input:  graph  $G(V, E)$ ; features  $X$ ; rates  $P = \{0, 0.1, \dots, 0.5\}$ ;
        seeds  $S = \{1..5\}$ 
Output: table  $R[p]$  (mean  $\pm$  std AUC per rate); retention metric
1:  $(E_{\text{train}}, E_{\text{val}}, E_{\text{test}}) \leftarrow \text{split}(E, 70/10/20)$ 
2: for each  $p$  in  $P$  do
3:   for each  $s$  in  $S$  do
4:      $E_{\text{obs}} \leftarrow \text{remove } \lfloor p \cdot |E_{\text{train}}| \rfloor \text{ random edges from } E_{\text{train}} \text{ (seed } s)$ 
5:      $\theta \leftarrow \text{train GraphSAGE + asymmetric MLP decoder on } (V, E_{\text{obs}}, X)$ 
6:      $R[p, s] \leftarrow \text{AUC}(\theta, E_{\text{test}} \cup \text{sampled negatives})$ 
7:   end for
8: end for
9: return mean, std of  $R$ ; retention  $\leftarrow \text{mean } R[0.5] / \text{mean } R[0]$ 

```

Algorithm 4.2 Disruption Signal Extraction and Linking (Module 4)

```

Input:  article text  $t$ ; uploaded graph  $G(V, E)$ ;
        mode  $m \in \{\text{auto}, \text{ollama}, \text{anthropic}, \text{rules}\}$ 
Output: signal  $s$  (event, severity, confidence, where, when,
        evidence, linked companies, exposure, method)
1: if  $m = \text{auto}$ :  $m \leftarrow \text{ollama}$  if server reachable,
   else anthropic if key set, else rules
2:  $s \leftarrow \text{LLM}_m(t, \text{JSON schema, names in } V) \triangleright \text{JSON mode, temp. } 0$ 
3: if the model call fails:  $s \leftarrow \text{RULES}(t)$ ; record reason
4: check each field of  $s$  against event / severity vocabulary;
   invalid value  $\rightarrow$  safe default
5:  $s.\text{companies} \leftarrow \{c \in s.\text{companies} : c \text{ appears in } t\} \triangleright \text{grounding}$ 
6: if  $s.\text{event} = \text{other}$ :  $s.\text{companies} \leftarrow \emptyset$ 
7: for each  $c$  in  $s.\text{companies}$  do
8:    $v \leftarrow \text{LINK}(c, V) \triangleright \text{exact} \rightarrow \text{token subset} \rightarrow \text{fuzzy}$ 
9:    $s.\text{exposure}[v] \leftarrow \text{direct buyers of } v, \text{ count two tiers down}$ 
10: end for
11:  $s.\text{method} \leftarrow \text{path that produced } s$ ; return  $s$ 

```

4.5 KEY FUNCTIONS

Table 4.2 Key functions of the Phase I implementation

Function	File	Description
load_records()	preprocess.py	Reads ETO / CSV records into a dataframe and drops incomplete rows
resolve_entities()	preprocess.py	Lower-cases names, applies alias table, merges duplicates
build_graph()	graph_builder.py	Creates directed supplies-to edge list, drops self-loops and repeats, assigns node ids
build_features()	graph_builder.py	One-hot node type and country, standardised in/out degree → matrix X
split_edges()	train.py	70/10/20 train/validation/test split with fixed test set
degrade_edges(p, seed)	train.py	Removes $\lfloor p \cdot E_{\text{train}} \rfloor$ random training edges
sage_forward()	gnn.py	Two-layer mean-aggregator GraphSAGE encoder
asym_decode(u, v)	gnn.py	Scores ordered pair via supplier/buyer projections
sample_negatives()	gnn.py	1:1 random non-links plus reversed true edges
train_model()	gnn.py	BCE loss, Adam, early stopping on validation AUC
eval_auc()	evaluate.py	AUC-ROC and average precision on held-out test edges
score_baselines()	evaluate.py	Jaccard and Adamic–Adar scores on the same candidates
retention()	evaluate.py	mean AUC@50% / mean AUC@0%
POST /upload	backend/main.py	Validates CSV, normalises tiers/countries, returns graph summary
POST /predict	backend/main.py	Trains per-upload model and returns ranked, tier-filtered predictions
TierGraph	frontend/	React component: tier-stratified network visualisation
chat_json()	signals/ollama.py	Calls Ollama /api/chat in JSON mode at temperature 0
_ollama_extract()	signals/extractor.py	Sends article and node names with the fixed schema; validates and grounds the reply
extract_signal()	signals/extractor.py	Mode switch (auto / ollama / anthropic / rules) with labelled fallback
extract_rules()	signals/rules.py	Offline lexicon extractor: event type, severity, company mentions
SupplyGraph.link()	signals/linker.py	Exact → token-subset → fuzzy entity linking to graph nodes
SupplyGraph.exposure()	signals/linker.py	Direct buyers and two-tier count for a linked node
POST /api/session/{id}/signals	signals/router.py	Extracts and links a pasted article for the session graph
POST /api/session/{id}/signals/news	signals/router.py	Gather → structure → link for one company in one call
GET /api/signals/status	signals/router.py	Reports whether Ollama is up, which model, and whether a NewsAPI key is set
SignalPanel	frontend/SignalPanel.jsx	React panel: paste or gather news → linked signal cards

4.6 MODULE DESCRIPTION

Table 4.3 Module ownership and status

No.	Module	Owner	Phase	Status
1	Data Ingestion & Graph Construction	Shankaranarayanan	I	Working – real ETO data
2	GNN Hidden Link Prediction	D. Sai Khushee	I	Working – AUC 0.845 on real data
3	Visibility-Degradation Robustness	D. Sai Khushee	I	Working – real ETO data
—	Tierline Web Platform	Both	I	Working – verified end to end
4	NLP Disruption Signal Extraction	Shankaranarayanan	I	Working – live news, Llama 3.1, integrated in Tierline
5	Multi-Source Corroboration & Alerting	D. Sai Khushee	II	Planned
6	Confidence-Gated Mitigation Recommender	Shankaranarayanan	II	Planned

4.6.1 Module 1 – Data Ingestion and Graph Construction

The module takes raw records, applies cleaning rules and builds the directed graph. Its process is: (i) load CSV records; (ii) assign an integer node id to each distinct company and product; (iii) merge duplicate names by lower-casing and an alias table; (iv) create one directed supplies-to edge for each record; (v) drop self-loops and repeated edges; (vi) save the graph in edge-list form. Every vertex is a company or product node and every edge means “supplies to”. The feature-engineering step then gives each node a fixed-length vector: a one-hot code for the node type (car maker, tier-1 supplier, chip maker, foundry, material), a one-hot country code, and standardised in- and out-degree. The feature matrix X of size $N \times d$ does not depend on any hidden edge, so it stays valid at every degradation level.

4.6.2 Module 2 – GNN Hidden Link Prediction

The encoder is a two-layer GraphSAGE network with mean aggregation. Layer k updates the embedding of node v as

$$h_v^{(k)} = \sigma \left(W^{(k)} \cdot \left[h_v^{(k-1)} \parallel \text{mean}\{h_u^{(k-1)} : u \in N(v)\} \right] \right) \quad (4.1)$$

where $N(v)$ is the set of visible neighbours of v , $\parallel$ means concatenation and σ is ReLU. GraphSAGE learns the aggregation function itself, not one fixed embedding per node, so it can still form sensible embeddings when parts of the neighbourhood are missing.

Supply chains are directed, so a symmetric decoder such as a dot product cannot tell “Bosch supplies Volkswagen” from “Volkswagen supplies Bosch”. The decoder

therefore projects each final embedding into two roles, supplier and buyer, and scores an ordered pair (u, v) as

$$\hat{y}_{uv} = \sigma(\text{MLP}(\mathbf{z}_u \parallel \mathbf{z}_v)) \quad (4.2)$$

Positive examples are the observed training edges. Negatives are randomly sampled non-links at a 1:1 ratio, resampled every epoch, together with the reversed copies of true edges, which forces the model to learn direction rather than mere association. The loss is binary cross-entropy,

$$\mathcal{L} = -\frac{1}{|B|} \sum_{(u,v) \in B} [y_{uv} \log \hat{y}_{uv} + (1 - y_{uv}) \log(1 - \hat{y}_{uv})] \quad (4.3)$$

optimised with Adam (learning rate 10^{-3}) and early stopping when validation AUC has not improved for 20 epochs.

4.6.3 Module 3 – Visibility-Degradation Robustness Protocol

This module is the core novelty of the system. It copies the situation a real manufacturer is in: part of the network is simply not known. Edges are first split into training, validation and test sets (70/10/20). The module then removes a fraction p of the training edges, where p ranges from 0% to 50% in steps of 10%, before the model sees them. The test set is fixed once and never touched, so every degradation level is scored against the same ground truth. For each level and each of five random seeds the model is retrained from scratch and scored by AUC-ROC and average precision. The retention metric is the mean AUC at 50% removal divided by the mean AUC at 0% removal – one number that states how much accuracy survives when half of the known links are hidden. The same protocol is run for the Jaccard and Adamic–Adar heuristics (and, in the Phase II sweep, node2vec and GCN).

4.6.4 Tierline Web Platform

Tierline is a React front end with a FastAPI back end. A company signs in, uploads a CSV of its known supplier relationships, confirms the graph the platform built from it (company count, relationship count, tier breakdown, country footprint), and then requests hidden-link prediction. The platform trains a fresh model on that upload alone, so there is no cold-start dependency on other companies' data and the file is processed in memory for the session only. The results page shows a tier-stratified network graph (vertical position = tier; solid lines = reported links; dashed lines = predicted links), a ranked table of predicted links with confidence and supporting evidence, a

concentration-risk panel that flags when a large share of the network sits in one country, and – since the Module 4 integration – a disruption signal panel for the same session. Predictions are filtered for tier plausibility, so the platform never suggests that an OEM supplies a chip maker. Table 4.4 gives the upload format; only two columns are required so a company can share as little as it is comfortable sharing.

Table 4.4 Tierline upload format

Column	Required	Example
supplier_name	Yes	Infineon Technologies
buyer_name	Yes	Robert Bosch
component_category	No	microcontroller
supplier_tier	No	Tier-2
supplier_country	No	DEU
buyer_tier	No	Tier-1
buyer_country	No	DEU

Tier values are normalised (T1, Tier1, tier-1 and Tier-1 all work) and missing tiers are kept as “Unknown” rather than dropped. The platform refuses files with fewer than 15 companies or 40 relationships rather than return a model that cannot generalise.

4.6.5 Module 4 – NLP Disruption Signal Extraction

Module 4 turns a news article into a structured disruption signal tied to the uploaded supply network. It is delivered as a Python package (backend/signals/) with its own FastAPI router, a React panel on the Tierline results page, a command-line demo runner and a test suite. The pipeline has five steps.

Gathering. For a company chosen from the uploaded network, the module queries NewsAPI for that name together with disruption terms and joins the title, description and body of each article into one text. The user can also paste any article or supplier notice directly into the panel.

Structuring. Each text is sent to Llama 3.1 (8B) running locally through Ollama, with a fixed JSON schema: event type (such as fire, earthquake, strike or export restriction, or “other” when the article reports no disruption), severity, confidence, where, when, a short evidence phrase and the companies named. The call uses Ollama’s JSON mode at temperature 0, so the same article always gives the same answer. Because the model runs on the team’s own machine, no supplier data leaves the computer and each

call costs nothing. If Ollama cannot be reached the module falls back to the Anthropic API when a key is set, and otherwise to a rule-based lexicon extractor; every result states which path produced it and, after a fallback, why.

Validation and grounding. Small local models still make mistakes, so every field is checked against the allowed event and severity values and a bad value is replaced with a safe default; stray reasoning tags and code fences are stripped. During testing a second, more serious fault appeared: for articles with no disruption, the model returned every company from the list of network names it had been given as a hint. The fix is a grounding rule – a company is kept only if its name actually appears in the article text, and all companies are cleared when the event type is “other”. After this change the non-disruption articles in the live run linked to no companies at all.

Entity linking. Company names in news rarely match the names in a supplier file. Each grounded name is matched to a graph node through a ladder of exact match, token-subset match and fuzzy match, so “Continental” links to Continental AG and “Denso” to DENSO.

Exposure. For every linked node the module lists the buyers that take supply from it directly and counts the buyers two tiers further down. The result is shown as a signal card: event, severity, confidence, the model that read the article, where, when and why, the companies named in the text next to their matched nodes, and who buys directly from them. This exposure list is the input that the Phase II corroboration module will score.

4.6.6 Phase II Modules (5 and 6)

Module 5 – Multi-Source Corroboration and Alerting cross-checks Module 4 signals from independent sources – NewsAPI articles and the GDELT event stream – and raises an alert only when they agree, with a confidence score, which removes single-source false alarms. **Module 6 – Confidence-Gated Mitigation Recommender** proposes responses (alternative suppliers, inventory buffers, re-routing), verifies each recommendation against the evidence it cites, and acts autonomously only when confidence exceeds a set threshold; otherwise it defers to a human. The modules and the predictor are integrated in one dashboard, validated by replaying the 2021–2022 chip shortage.

4.7 MEASURED RESULTS ON THE REAL ETO GRAPH (MODULES 1–3)

Modules 1–3 were run end to end on the processed ETO graph of 282 companies and 720 relationships using the protocol of Algorithm 4.1 with five seeds per level. Table 4.5 gives the measured values at full visibility and at the hardest level, 50% removal; the values for the intermediate levels (10–40%) are recorded in `real_results_table.csv` produced by `real_experiment.py`. Figure 4.1 plots the same end points.

Table 4.5 Measured results on the real ETO graph (NumPy prototype, mean of 5 seeds)

Edges removed	GraphSAGE AUC	CF-Jaccard AUC	CF-Adamic-Adar AUC	GraphSAGE retention
0%	0.845	0.760	0.764	1.00
50%	0.787	≈0.64	≈0.64	0.93

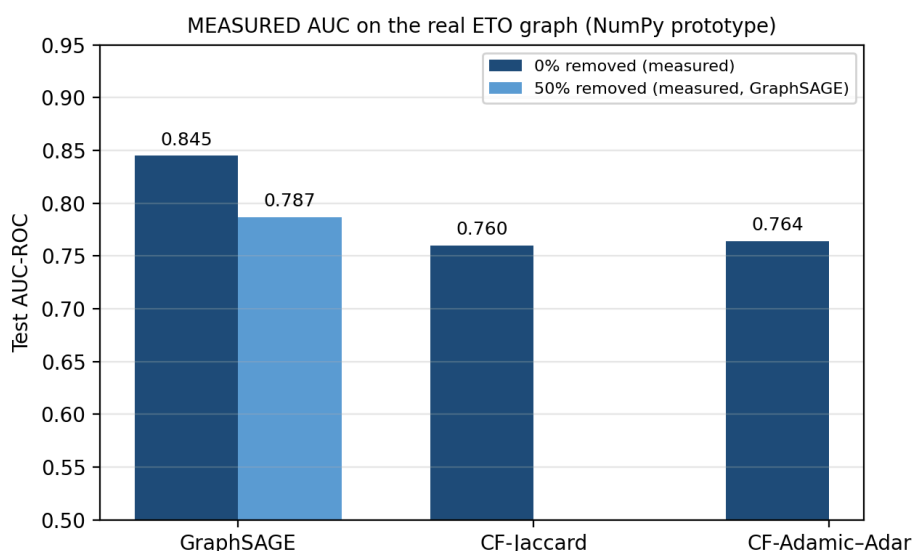


Figure 4.1 Measured test AUC on the real ETO graph (NumPy prototype)

At full visibility GraphSAGE reaches an AUC of 0.845, which is 0.08 above the better of the two heuristic baselines. When half of the training edges are removed, GraphSAGE still scores 0.787, a retention of 93%, while the gap over the heuristics widens to about 0.15. The GNN therefore degrades more slowly than the neighbourhood heuristics, which is the behaviour the visibility-degradation protocol was built to detect. Two points must be read alongside these numbers. First, the prototype is a NumPy implementation with modest hyper-parameter tuning; the Phase II PyTorch Geometric version is expected to improve absolute AUC. Second, as noted in Section 4.1, the ETO graph links companies to processes and tools, so this experiment validates the method on real published supply data but not yet on firm-to-firm automotive relationships.

4.8 EFFECT OF THE ASYMMETRIC DECODER (TIERLINE)

The first version of the decoder scored a pair with a dot product, which is symmetric: $\text{score}(u, v) = \text{score}(v, u)$. On the Tierline sample network the model could tell that two firms were related but not which way goods flowed, and it produced predictions such as “Volkswagen supplies Bosch”. Projecting each company into separate supplier and buyer roles and training on reversed edges as negatives fixed this. On the sample network (training on 123 relationships, testing on 26 held-out ones) direction accuracy rose from 0.500, i.e. chance, to 1.000 and test AUC rose from 0.679 to 0.880, against 0.34 for the Adamic–Adar heuristic on the same test set (Figure 4.2). These numbers demonstrate the workflow on constructed edges and are not an empirical claim about real automotive relationships.

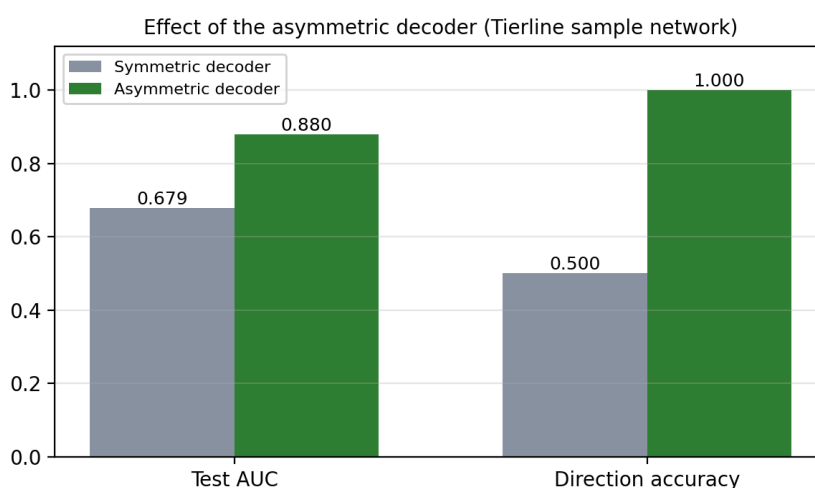


Figure 4.2 Effect of the asymmetric decoder on the Tierline sample network

4.9 DISRUPTION SIGNAL EXTRACTION RESULTS (MODULE 4)

Module 4 was run inside the live Tierline platform on 8 September 2026 on a MacBook Air (Apple M3), with the automotive sample network uploaded. A NewsAPI query for Renesas Electronics returned five real articles, and each was read by llama3.1 through Ollama. The two bundled sample articles that carry a fire and a no-disruption control were run through the same path. Table 4.6 lists every result.

Table 4.6 Module 4 results on live news and bundled samples

Article (source, date)	Model output	Severity / confidence	Linked to network
Renesas Kumamoto plant halted after M7.1 earthquake (Biztoc, 11 Aug)	Earthquake	High / 100%	Renesas Electronics → 5 direct buyers, 10 more two tiers down
Consumer-tech news item (CNET, 18 Aug)	No disruption found	— / 20%	—
Renesas stock recovered since 2016 quake (BeInCrypto, 11 Aug)	No disruption found (historic)	— / 50%	—
Stoneridge launches new ECU platform (PRNewswire, 10 Aug)	No disruption found	— / 10%	—
History of a Japanese electronics firm (Medium, 7 Aug)	No disruption found	— / 20%	—
Bundled sample: Naka fab fire (written by the team)	Fire	Critical / 100%	Renesas Electronics, DENSO, Continental AG → buyers
Bundled control: quarterly results (written by the team)	No disruption found	— / 20%	None (correct)

Of the five live articles, exactly one described a current disruption – a halt at the Renesas Kumamoto plant after a magnitude 7.1 earthquake – and the model classed it as an earthquake of high severity, named Japan as the location, quoted the halt as evidence and linked Renesas Electronics to its node in the uploaded network. The card listed the five direct buyers of that node (Aisin Corporation, Hyundai Mobis, Magna International, Panasonic Automotive and ZF Friedrichshafen) and ten more two tiers down. The other four articles – a consumer-tech item, a stock story, a product launch and a company history – were all labelled as no disruption. The stock story mentions the 2016 Kumamoto earthquake, and the model correctly treated it as old news rather than a current event. On the bundled fire sample, the linker matched the article’s “Continental” and “Denso” to the network nodes Continental AG and DENSO.

These are functional checks, not an accuracy figure: no labelled test set of real headlines exists yet, seven articles are far too few to estimate precision or recall, and the exposure lists come from the constructed sample network described in Section 4.1. Building a hand-labelled set of real headlines and reporting precision and recall for event type and company linking is a Phase II task.

4.10 PROJECTED RESULTS FOR THE FULL FIVE-METHOD SWEEP

The values in this section are projections anchored in published GraphSAGE performance on a comparable automotive supply network [9]. They state the outcome the experimental design is built to test and will be replaced by measured values once the Phase II PyTorch runs (six drop rates $\times$ five seeds $\times$ five methods) are complete.

Table 4.7 Projected test AUC under increasing visibility degradation (NOT measured)

Method	0%	10%	20%	30%	40%	50%	Retention
Jaccard	0.78	0.72	0.66	0.60	0.54	0.50	0.64
Adamic-Adar	0.81	0.76	0.70	0.63	0.57	0.52	0.64
node2vec	0.84	0.80	0.75	0.70	0.64	0.58	0.69
GCN	0.88	0.85	0.82	0.78	0.73	0.68	0.77
GraphSAGE (proposed)	0.91	0.89	0.87	0.84	0.80	0.76	0.84

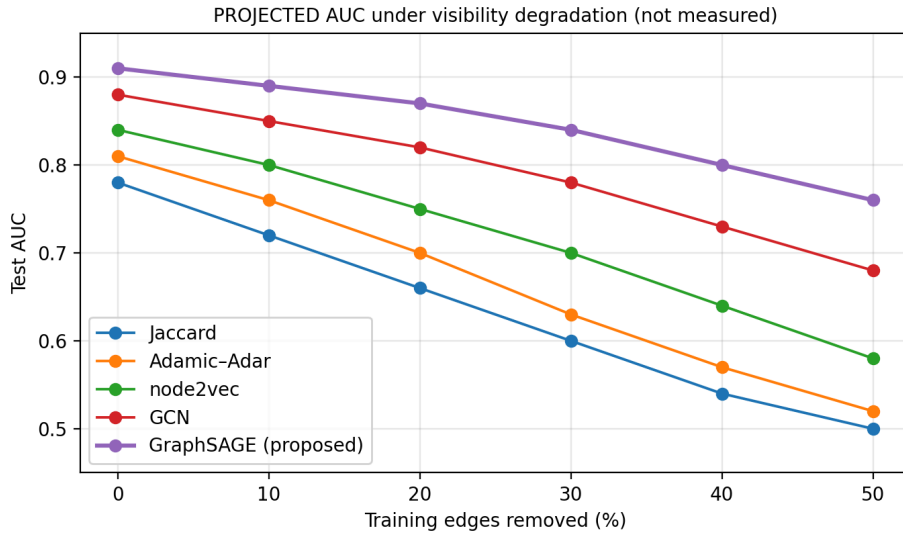


Figure 4.3 Projected AUC degradation curves for all methods (not measured)

Figure 4.3 shows the projected behaviour of each method as visibility falls. The two heuristics are expected to decline steeply and approach the random baseline of 0.50 at 50% removal, because the neighbourhood overlap they rely on disappears together with the deleted edges. node2vec should hold up slightly better, since its random walks capture some longer-range structure, but still lose about a third of its accuracy. The two GNNs

should degrade far more gently, and GraphSAGE, which learns an inductive aggregation function, is projected to retain roughly 84% of its full-visibility AUC when half of the training edges are hidden. The measured ETO prototype result of 93% retention (Section 4.7) is consistent with this projection. The practical reading is direct: beyond roughly 30% visibility loss, heuristic scores stop being trustworthy, while the proposed model still produces rankings a purchasing team at a car maker could act on.

4.11 PLATFORM VERIFICATION

Tierline was verified end to end with Playwright browser automation: sign-in, sample-network load, graph confirmation, prediction and results rendering, with zero JavaScript errors from a clean install. Module 4 has 15 automated tests covering the rule-based path, entity linking, the Ollama path against a fake local server, the fallback to rules when the model is missing, the NewsAPI fetcher and every API endpoint; all 15 pass. The real Ollama and NewsAPI paths were then run on the team's laptop inside the live platform (Section 4.9). Known limitations of the Phase I platform are in-memory sessions, stub authentication, a cap of roughly 3,000 companies per upload, the NumPy model in place of PyTorch, NewsAPI free-plan limits (about 200 characters of body text and a 24-hour delay), and one signal per article, so an article that reports two separate events yields only one. Figures 4.4 to 4.7 show the login, upload, confirmation and results pages.

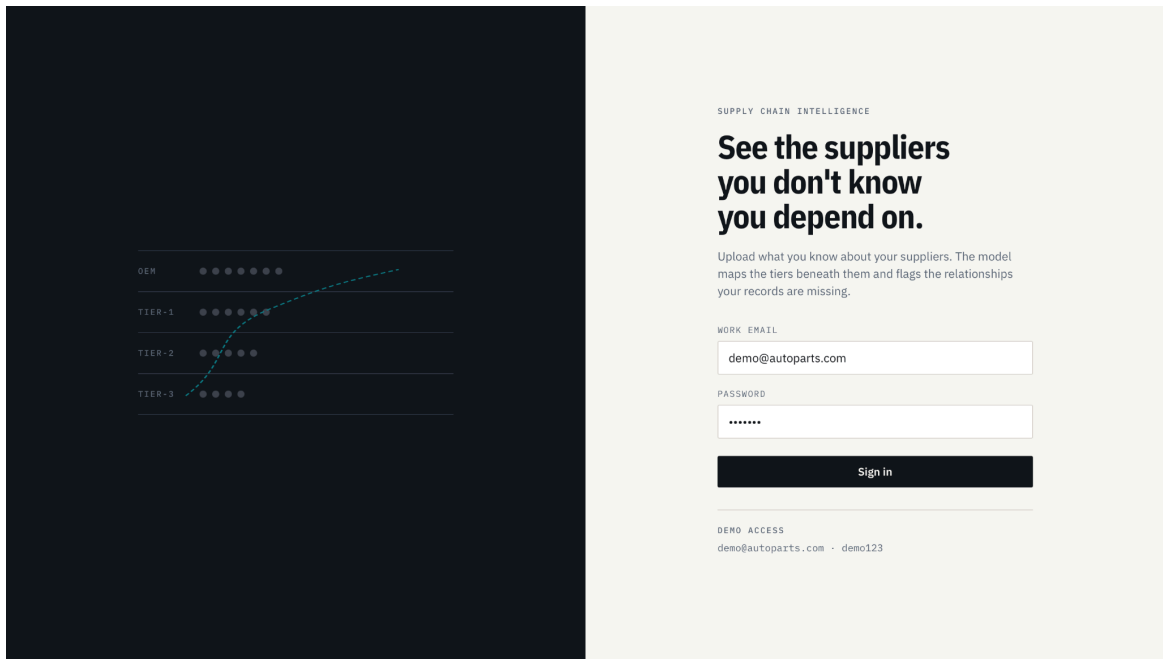


Figure 4.4 Tierline platform screens – LOGIN

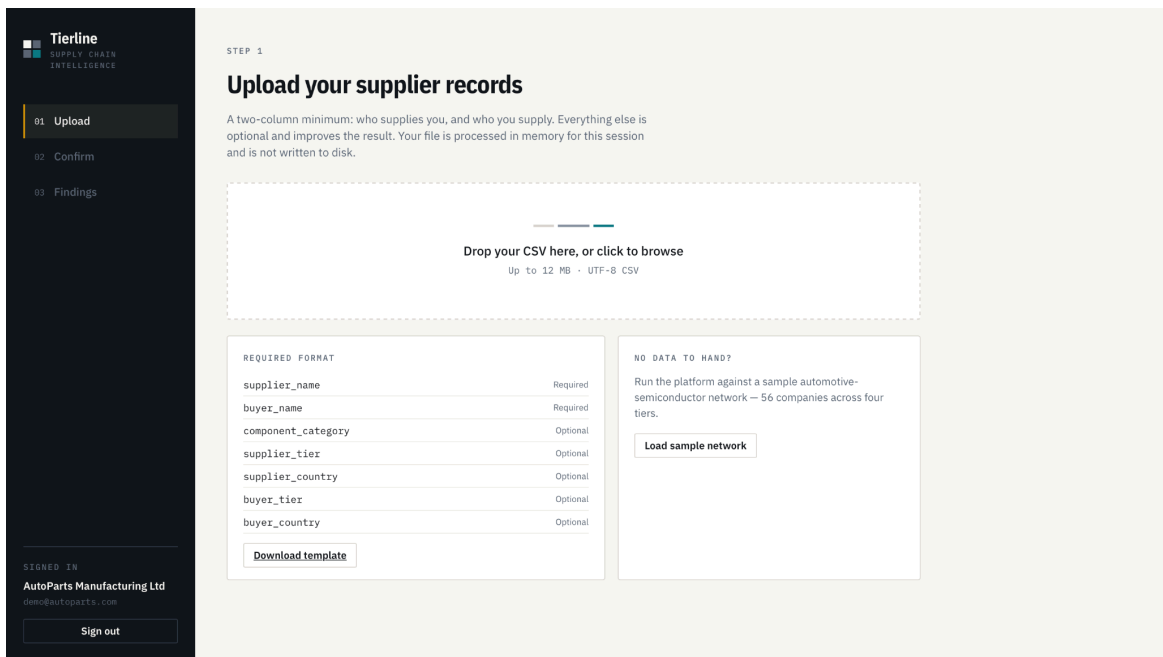


Figure 4.5 Tierline platform screens – UPLOAD

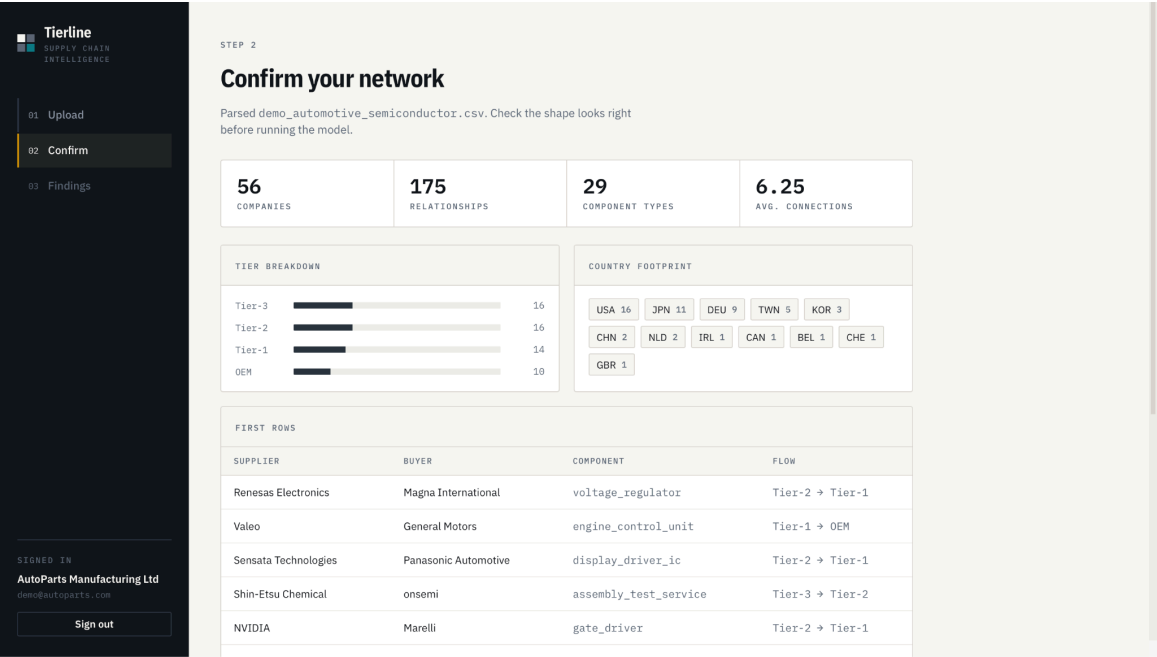


Figure 4.6 Tierline platform screens – CONFIRM

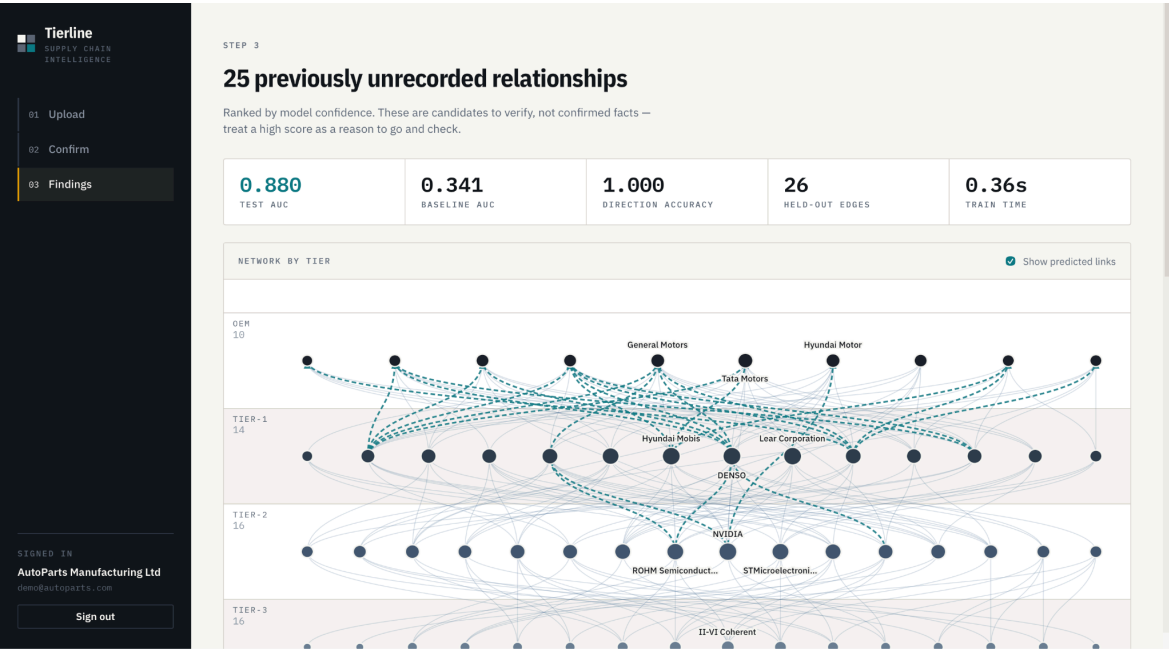


Figure 4.7 Tierline platform screens – RESULTS (a)

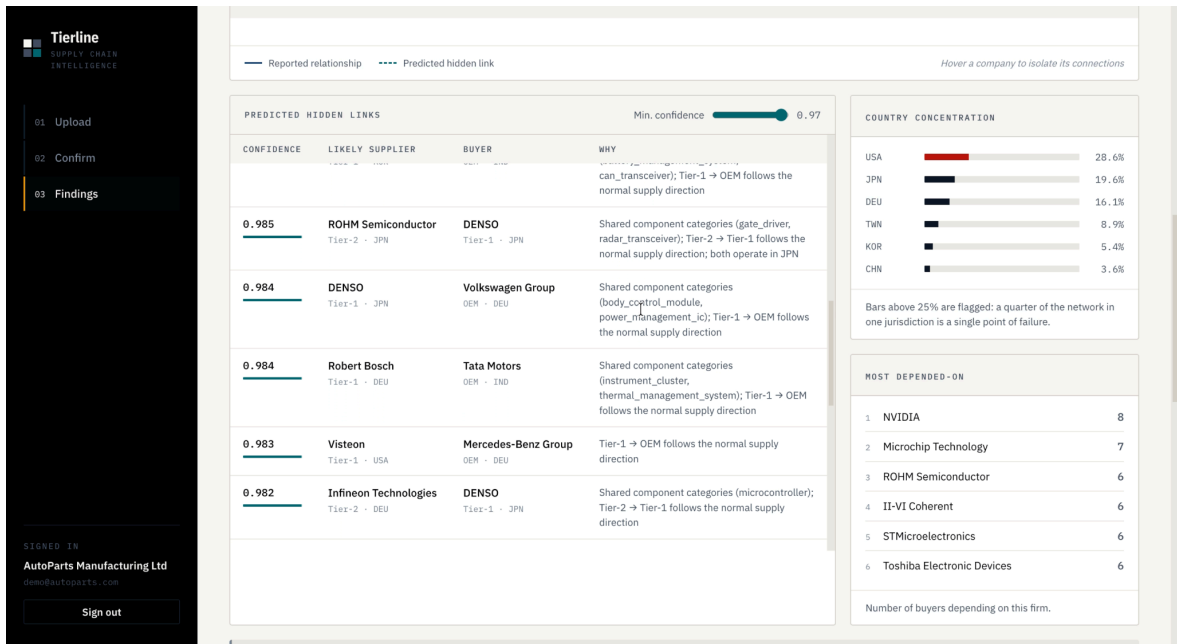


Figure 4.8 Tierline platform screens – RESULTS (b)

Tierline
SUPPLY CHAIN INTELLIGENCE

01 Upload
02 Confirm
03 Findings

SIGNED IN
AutoParts Manufacturing Ltd
demo@autoparts.com
Sign out

How to read these numbers. The model was trained only on the 123 relationships in your file, then tested on 26 it never saw. Test AUC 0.880 means it ranked a real held-out relationship above a random pair 88% of the time. The baseline (0.341) is Adamc-Adar, a standard non-learned heuristic, on the same test set.

Disruption signals

Paste a news article or supplier notice. The system reads it, names the event, and shows which companies in your uploaded network are affected.

Try a sample

Fire halts output at Renesas automotive chip fab Magnitude 6.8 earthquake shakes Hsinchu, fabs evacuated

Dockworkers begin indefinite strike at Port of Rotterdam

New export licence rules cover automotive-grade power semiconductors

Control article: quarterly results, no disruption

Volk Gather recent news

Reads with llama3.1 running locally via Ollama. Nothing leaves this machine.

Paste article text here

Clear Extract signal

Figure 4.9 Tierline platform screens – Disruption Signal Extraction

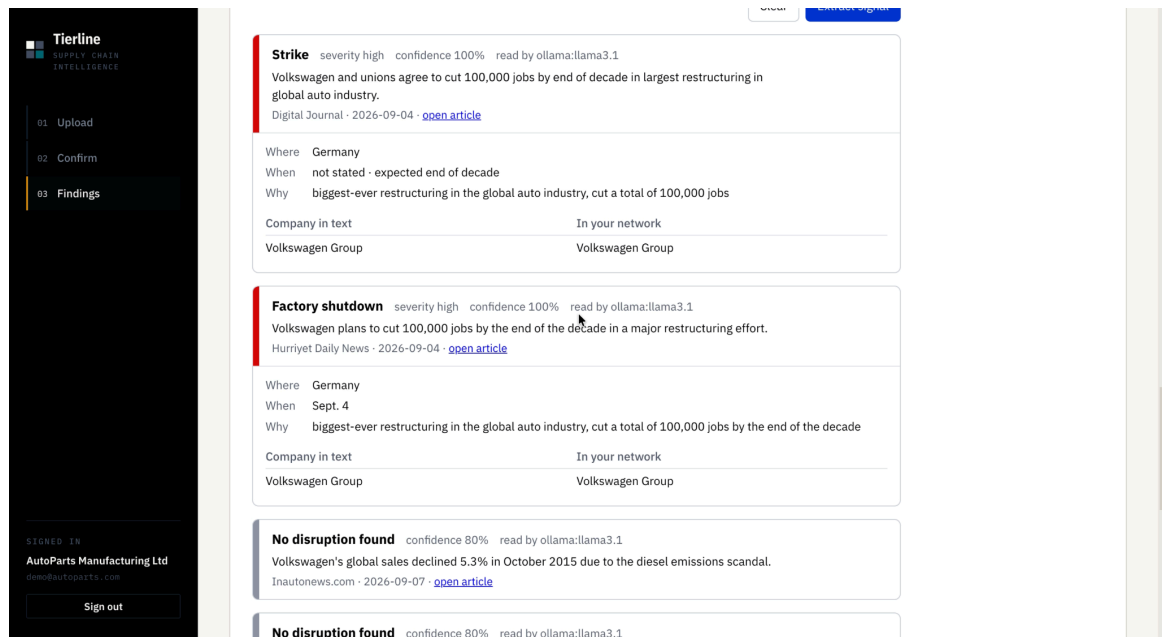


Figure 4.10 Tierline platform screens – NewsAPI Output

4.12 DISCUSSION

Phase I answers the question it set out to ask: on real published supply data, a GraphSAGE link predictor keeps most of its accuracy when half of the known links are hidden, and it holds up much better than the neighbourhood heuristics that existing work uses as baselines. The retention metric turns the degradation curve into one number that can be quoted when comparing methods. The direction fix in the decoder is a practical finding: any link predictor applied to supply chains must be asymmetric, or it will produce reversed relationships that a purchasing team cannot use.

Module 4 adds a second practical finding. A small language model running on a laptop is good enough to read supply chain news and fill a fixed schema, which keeps the cost at zero and the data on site, but it cannot be trusted on its own: without the grounding rule it attached every company in the network to articles that reported nothing. Checking each extracted name against the source text, as the hallucination literature recommends [22], removed this fault in the live run. The main open issues are data and evaluation. Real firm-to-firm automotive relationships are commercial, so the automotive layer of the graph is built from public disclosures and, in the demo, from a constructed sample; and Module 4 still needs a labelled set of headlines before an accuracy number can be reported. Phase II will address the first by mining links from news in the style of [8] and

by obtaining any partner-company data that becomes available, and the second by building that labelled set.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

This report argued that a supply chain link predictor must be evaluated on incomplete graphs, because incomplete graphs are the only kind that exist in practice. Phase I delivered a directed automotive electronics supply graph built from public CSET/ETO data and public supplier records, a GraphSAGE link predictor with a direction-aware decoder, a visibility-degradation protocol with a retention metric, and the Tierline web platform through which a company can run the predictor on its own supplier list. Measured on the real ETO graph, the model reaches an AUC of 0.845 and retains 93% of it when half of the known links are hidden, ahead of the Jaccard and Adamic–Adar baselines. Phase I also delivered Module 4, which reads news with a locally run Llama 3.1 model, checks every extracted company against the article, and links the result to the uploaded network; on live news it picked out the one real disruption among five articles and showed the buyers exposed to it. The Phase I research paper has been accepted at SMART-2026.

5.2 FUTURE WORK – PHASE II PLAN

Phase II (8th semester, 6 credits) completes the platform across three reviews.

1. **Module 5 – Multi-Source Corroboration and Alerting.** Add the GDELT event stream as a second independent source next to NewsAPI, and raise an alert only when independent sources agree, with a confidence score; evaluate false-alarm reduction on the 2021–2022 chip shortage replay.
2. **Module 6 – Confidence-Gated Mitigation Recommender.** Generate mitigation recommendations, verify each against cited evidence, and gate autonomous action by a confidence threshold.
3. **Module 4 evaluation and upgrades.** Hand-label a set of real headlines and report precision and recall for event type and company linking; replace the 200-character NewsAPI snippets with full-text article retrieval; allow more than one signal per article.

4. **Full GNN sweep.** Run the PyTorch Geometric sweep of all five methods (six drop rates $\times$ five seeds) and replace every projected value in this report with a measured one.
5. **Better degradation model.** Remove deep-tier edges more often than tier-1 edges instead of removing edges uniformly, which matches how visibility is actually lost.
6. **Richer graph.** Merge links mined from news in the style of Wichmann et al. [8] into the graph and test whether text-mined edges recover lost accuracy; obtain firm-level automotive data where possible.
7. **Integration.** Tie the predictor, signal extraction, alerting and mitigation modules into one dashboard so that identified hidden links feed disruption risk scoring directly; harden the platform (persistent sessions, real authentication, PyTorch model).
8. **Paper 2.** Publish the Phase II paper covering the alerting and mitigation modules, sharing the common platform abstract.

REFERENCES

- [1]D. Liben-Nowell and J. Kleinberg, “The link-prediction problem for social networks,” *Journal of the American Society for Information Science and Technology*, vol. 58, no. 7, pp. 1019–1031, 2007.
- [2]A. Grover and J. Leskovec, “node2vec: Scalable feature learning for networks,” in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 855–864.
- [3]T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.
- [4]W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [5]P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2018.
- [6]M. Zhang and Y. Chen, “Link prediction based on graph neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [7]G. Baryannis, S. Validi, S. Dani, and G. Antoniou, “Supply chain risk management and artificial intelligence: state of the art and future research directions,” *International Journal of Production Research*, vol. 57, no. 7, pp. 2179–2202, 2019.
- [8]P. Wichmann, A. Brintrup, S. Baker, P. Woodall, and D. McFarlane, “Extracting supply chain maps from news articles using deep neural networks,” *International Journal of Production Research*, vol. 58, no. 17, pp. 5320–5336, 2020.
- [9]E. E. Kosasih and A. Brintrup, “A machine learning approach for predicting hidden links in supply chain with graph neural networks,” *International Journal of Production Research*, vol. 60, no. 17, pp. 5380–5393, 2022.
- [10]A. Aziz, E. E. Kosasih, R.-R. Griffiths, and A. Brintrup, “Data considerations in graph representation learning for supply chain networks,” *arXiv preprint arXiv:2107.10609*, 2021.
- [11]W. Mohammad, A. Elomri, and L. Kerbache, “The global semiconductor chip shortage: Causes, implications, and potential remedies,” *IFAC-PapersOnLine*, vol. 55, no. 10, pp. 476–483, 2022.
- [12]Emerging Technology Observatory, “Supply Chain Explorer: Advanced Chips,” Center for Security and Emerging Technology (CSET), Georgetown University. [Online]. Available: <https://eto.tech> and <https://github.com/georgetown-cset/eto-chip-explorer>
- [13]T. N. Kipf and M. Welling, “Variational graph auto-encoders,” *arXiv preprint arXiv:1611.07308*, 2016.
- [14]Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, “A comprehensive survey on graph neural networks,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, 2021.

- [15]A. Brintrup, P. Wichmann, P. Woodall, D. McFarlane, E. Nicks, and W. Krechel, “Predicting hidden links in supply networks,” *Complexity*, vol. 2018, Article ID 9104387, 2018.
- [16]A. Brintrup, J. Pak, D. Ratiney, T. Pearce, P. Wichmann, P. Woodall, and D. McFarlane, “Supply chain data analytics for predicting supplier disruptions: a case study in complex asset manufacturing,” *International Journal of Production Research*, vol. 58, no. 11, pp. 3330–3341, 2020.
- [17]E. E. Kosasih, F. Margaroli, S. Gelli, A. Aziz, N. Wildgoose, and A. Brintrup, “Towards knowledge graph reasoning for supply chain risk management using graph neural networks,” *International Journal of Production Research*, vol. 62, no. 15, pp. 5596–5612, 2024.
- [18]D. Ivanov and A. Dolgui, “Viability of intertwined supply networks: extending the supply chain resilience angles towards survivability,” *International Journal of Production Research*, vol. 58, no. 10, pp. 2904–2915, 2020.
- [19]D. Ivanov, “Predicting the impacts of epidemic outbreaks on global supply chains: A simulation-based analysis on the coronavirus outbreak (COVID-19/SARS-CoV-2) case,” *Transportation Research Part E: Logistics and Transportation Review*, vol. 136, 101922, 2020.
- [20]K. Leetaru and P. A. Schrod, “GDELT: Global data on events, location, and tone, 1979–2012,” in *ISA Annual Convention*, San Francisco, 2013.
- [21]A. Dubey et al., “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [22]Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

CONFERENCE/JOURNAL PAPER PUBLICATION

CURRENT SUBMISSION DETAILS

Conference Name: 15th IEEE Scopus Indexed International Conference on System Modelling & Advancement on Research Trends (**SMART-2026**)

Status: Accepted

Paper ID: 434

Submitted on: 20/09/2026

Paper Title: Robust Prediction of Hidden Supplier Links in Automotive Electronics Supply Chains Using Graph Neural Networks

Abstract: Car production depends on electronic parts that pass through several layers of suppliers. A car maker generally just knows its direct suppliers, so the chip makers, foundries and material vendors below them remain invisible to them. The 2021–2022 chip shortage showed the cost of this blind spot: many car plants stopped work because of parts they had not realised they relied on originally. Big companies buy expensive mapping and monitoring platforms specialized to their supply chain whereas small and mid-sized automotive electronics manufacturers can't buy or create such systems. This paper constructs an AI platform to provide them with the same protection at low cost. The platform consists of 3 parts. First, a graph neural network predicts hidden supplier links from partial data, and its robustness is tested by deleting known links step by step (10–50%) and measuring accuracy at each level. Second, a disruption alerting module watches global event data and raises an alert only after multiple independent sources confirm the same event, which cuts out the false alarms. Third, a mitigation module recommends responses, checks every recommendation against the evidence it cites, and acts on its own only when its confidence crosses a set threshold. Projected results indicate the prediction model keeps about 84% of its accuracy even when half of the known links are hidden. The platform is built on public data, the CSET/ETO chip supply dataset, public supplier records and the GDELT event database, and is validated by replaying the 2021–2022 chip shortage.

Authors:

- Dr. P. Kumar [kumar@rajalakshmi.edu.in] (Primary)
- D. Sai Khushee [230701276@rajalakshmi.edu.in]
- Shankaranarayanan [230701306@rajalakshmi.edu.in]